

Apostila de Matplotlib

Visualização de Dados em Python — do zero ao gráfico

Conteúdo: preparação de dados, anatomia da figura, gráficos de linha, barras, pizza, dispersão, histograma, boxplot, subplots e personalização.

Sumário

- 1. Introdução ao Matplotlib
- 2. Instalação e Configuração
- 3. Anatomia de uma Figura
- 4. Preparação de Dados
- 5. Gráfico de Linha
- 6. Gráfico de Barras
- 7. Barras Agrupadas
- 8. Gráfico de Pizza
- 9. Gráfico de Dispersão
- 10. Histograma
- 11. Boxplot
- 12. Subplots (Múltiplos Gráficos)
- 13. Matplotlib + Pandas
- 14. Personalização e Estilo
- 15. Salvando Gráficos
- 16. Conclusão

1. Introdução ao Matplotlib

Matplotlib é a biblioteca mais popular do ecossistema Python para criação de gráficos estáticos, animados e interativos. Ela é flexível, integra-se nativamente com NumPy e Pandas, e serve como base para outras bibliotecas como Seaborn e Pandas plotting.

Nesta apostila vamos partir do básico — preparação dos dados — até a geração e personalização de diversos tipos de gráficos. Cada seção contém o **código completo** e a **imagem do gráfico gerado**.

2. Instalação e Configuração

Instale via pip no terminal:

```
pip install matplotlib numpy pandas
```

Em scripts Python, o módulo principal é importado com o apelido padrão `plt`:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

3. Anatomia de uma Figura

Uma figura do Matplotlib é composta por:

- **Figure**: o quadro inteiro (a janela ou imagem final).
- **Axes**: a área onde o gráfico é desenhado (cada gráfico é um Axes).
- **Axis**: os eixos X e Y, com escalas e rótulos.
- **Artists**: tudo que aparece (linhas, textos, legendas, títulos).

Existem duas formas principais de uso:

```
# 1) Interface pyplot (rápida)
plt.plot([1,2,3],[4,5,6])
plt.title('Exemplo')
plt.show()

# 2) Interface orientada a objetos (recomendada)
fig, ax = plt.subplots()
ax.plot([1,2,3],[4,5,6])
ax.set_title('Exemplo')
plt.show()
```

4. Preparação de Dados

Antes de plotar, é fundamental organizar os dados. Os formatos mais usados são **listas Python**, **arrays NumPy** e **DataFrames Pandas**.

4.1 Listas

```
meses = ['Jan', 'Fev', 'Mar', 'Abr']  
vendas = [120, 150, 90, 180]
```

4.2 Arrays NumPy

NumPy é ideal para dados numéricos e cálculos vetorizados.

```
import numpy as np  
x = np.linspace(0, 10, 100) # 100 pontos entre 0 e 10  
y = np.sin(x)               # aplica seno em cada ponto
```

4.3 DataFrames Pandas

Pandas é a melhor opção para dados tabulares (planilhas, CSVs).

```
import pandas as pd  
df = pd.DataFrame({  
    'mes': ['Jan', 'Fev', 'Mar'],  
    'vendas': [120, 150, 90]  
})  
print(df.head())
```

Dica: sempre verifique tipos (`df.dtypes`), valores nulos (`df.isna().sum()`) e ordene/filtre antes de plotar.

5. Gráfico de Linha

Ideal para mostrar a **evolução de uma variável** ao longo do tempo ou de uma sequência contínua.

```
import matplotlib.pyplot as plt
import numpy as np

# 1) Preparar os dados
x = np.linspace(0, 2*np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# 2) Criar o gráfico
plt.plot(x, y1, label='seno', linewidth=2)
plt.plot(x, y2, label='cosseno', linestyle='--')

# 3) Decorar
plt.title('Funções Trigonométricas')
plt.xlabel('x (radianos)')
plt.ylabel('f(x)')
plt.legend()
plt.show()
```

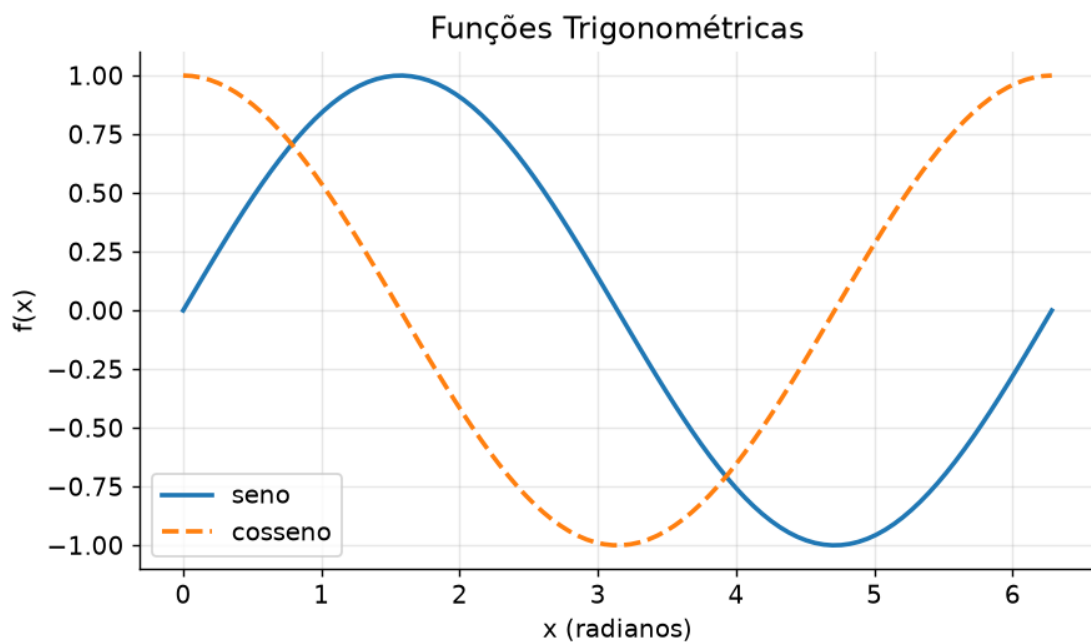


Figura 1 — Gráfico de linha com duas séries.

6. Gráfico de Barras

Compara **valores entre categorias discretas**. Use `plt.bar` (vertical) ou `plt.barh` (horizontal).

```
categorias = ['Jan', 'Fev', 'Mar', 'Abr', 'Mai', 'Jun']
vendas = [120, 150, 90, 180, 210, 175]

bars = plt.bar(categorias, vendas, color='#3B82F6', edgecolor='#1E40AF')

# Adicionar valor em cima de cada barra
for b, v in zip(bars, vendas):
```

```
plt.text(b.get_x()+b.get_width()/2, v+3, str(v),  
        ha='center', fontsize=9)  
  
plt.title('Vendas por Mês')  
plt.ylabel('Unidades')  
plt.show()
```

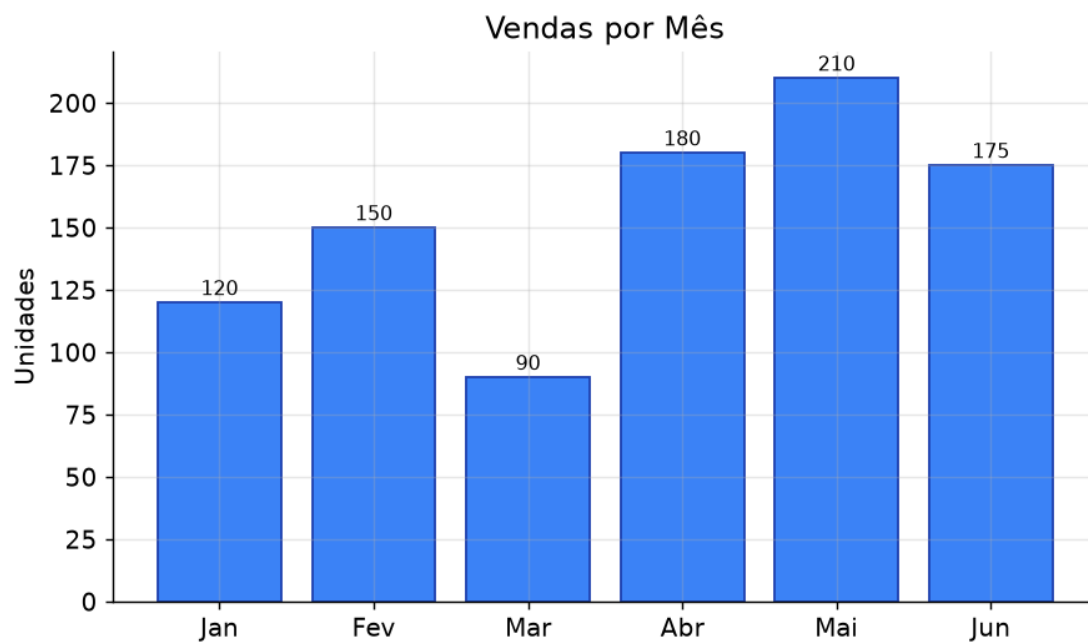


Figura 2 — Gráfico de barras com rótulos.

7. Barras Agrupadas

Para comparar **várias séries** nas mesmas categorias, deslocamos as barras com NumPy.

```
import numpy as np

prod_a = [120, 150, 90, 180, 210, 175]
prod_b = [100, 130, 110, 160, 190, 200]
x = np.arange(len(categorias))
w = 0.4

plt.bar(x - w/2, prod_a, w, label='Produto A', color='#3B82F6')
plt.bar(x + w/2, prod_b, w, label='Produto B', color='#F97316')
plt.xticks(x, categorias)
plt.legend(); plt.title('Comparação de Vendas')
plt.show()
```

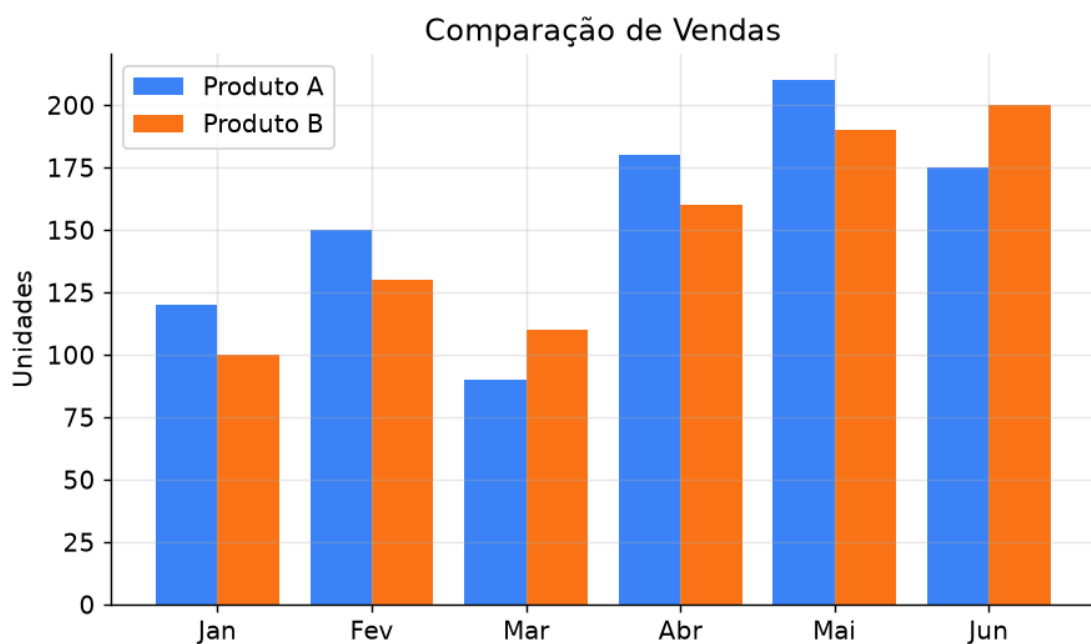


Figura 3 — Barras agrupadas para comparação.

8. Gráfico de Pizza

Mostra a **proporção** de cada parte em relação ao todo. Use para poucas categorias (até ~6).

```
labels = ['Aluguel', 'Comida', 'Transporte', 'Lazer', 'Outros']
valores = [1200, 800, 400, 300, 250]

plt.pie(valores, labels=labels, autopct='%1.1f%%',
        startangle=90,
        colors=['#3B82F6', '#10B981', '#F59E0B', '#EF4444', '#8B5CF6'])
plt.title('Distribuição de Gastos Mensais')
plt.show()
```

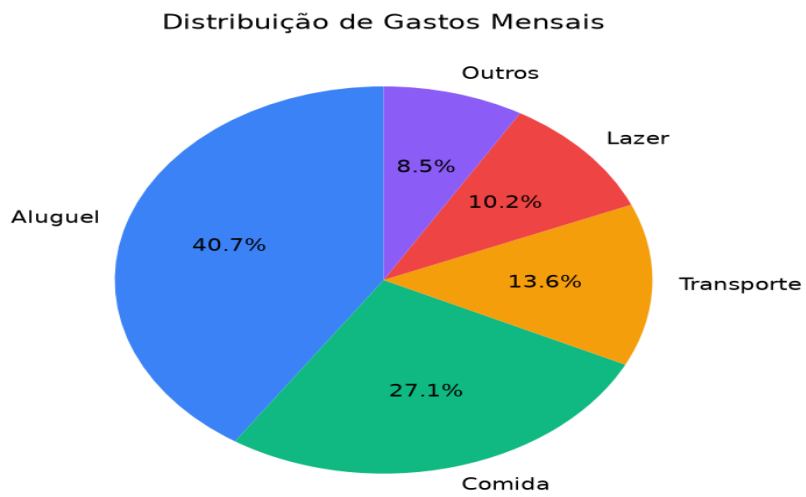


Figura 4 — Pizza com porcentagens automáticas.

9. Gráfico de Dispersão (Scatter)

Mostra a **relação entre duas variáveis numéricas**. Útil para identificar correlações e outliers.

```
rng = np.random.default_rng(42)
horas = rng.uniform(1, 10, 50)
notas = 2*horas + rng.normal(0, 2, 50) # ruído

plt.scatter(horas, notas, c=notas, cmap='viridis',
            s=60, edgecolor='k')
plt.colorbar(label='Nota')
plt.xlabel('Horas de Estudo'); plt.ylabel('Nota')
plt.title('Horas de Estudo x Nota')
plt.show()
```

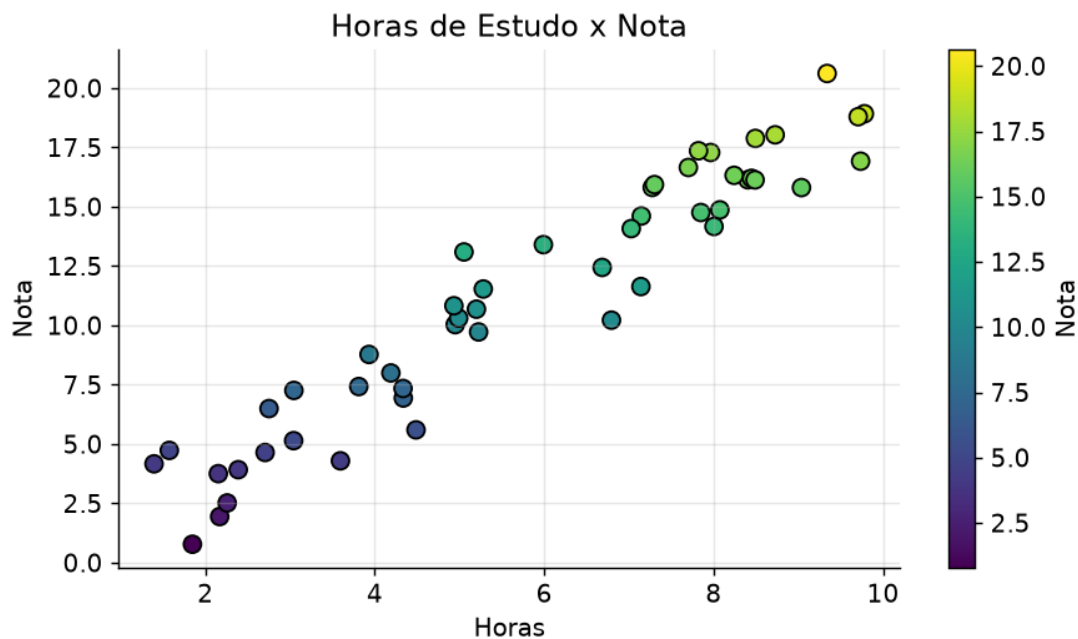


Figura 5 — Dispersão com mapa de cores.

10. Histograma

Mostra a **distribuição de frequência** de uma variável contínua, dividindo os dados em *bins*.

```
dados = rng.normal(170, 8, 1000) # altura em cm

plt.hist(dados, bins=30, color='#10B981', edgecolor='white')
plt.axvline(dados.mean(), color='red', linestyle='--',
            label=f'Média={dados.mean():.1f}')
plt.xlabel('Altura (cm)'); plt.ylabel('Frequência')
plt.title('Distribuição de Alturas')
plt.legend(); plt.show()
```

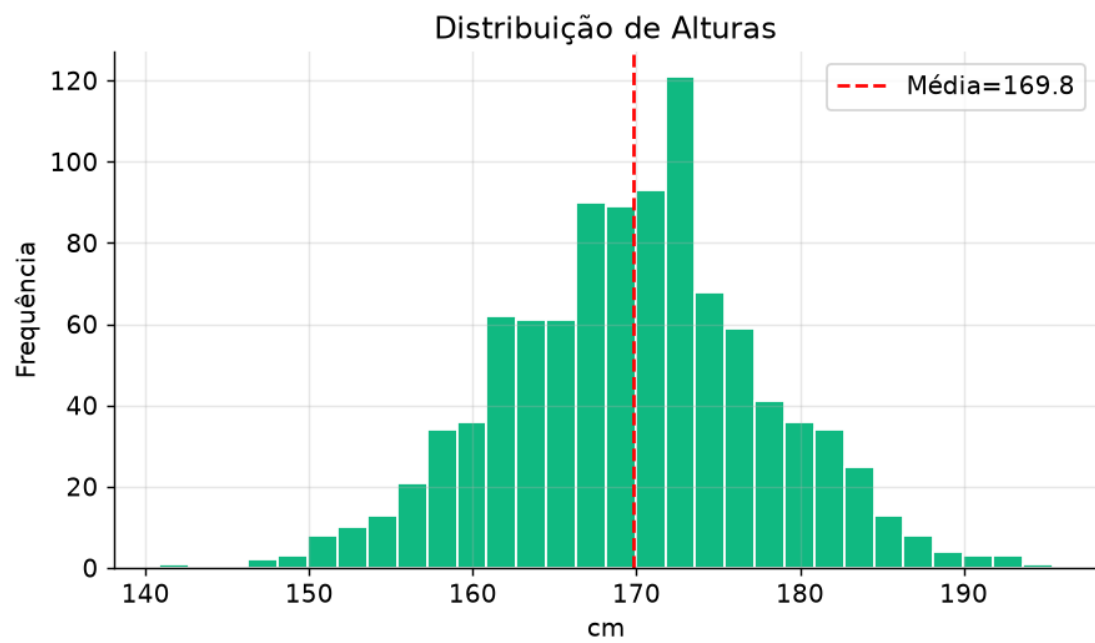


Figura 6 — Histograma com linha de média.

11. Boxplot

Resume a distribuição mostrando **mediana, quartis e outliers**. Excelente para comparar grupos.

```
g1 = rng.normal(70, 10, 100)
g2 = rng.normal(75, 8, 100)
g3 = rng.normal(65, 12, 100)

plt.boxplot([g1, g2, g3],
            labels=['Turma A', 'Turma B', 'Turma C'],
            patch_artist=True,
            boxprops=dict(facecolor='#93C5FD'))
plt.ylabel('Nota')
plt.title('Notas por Turma')
plt.show()
```

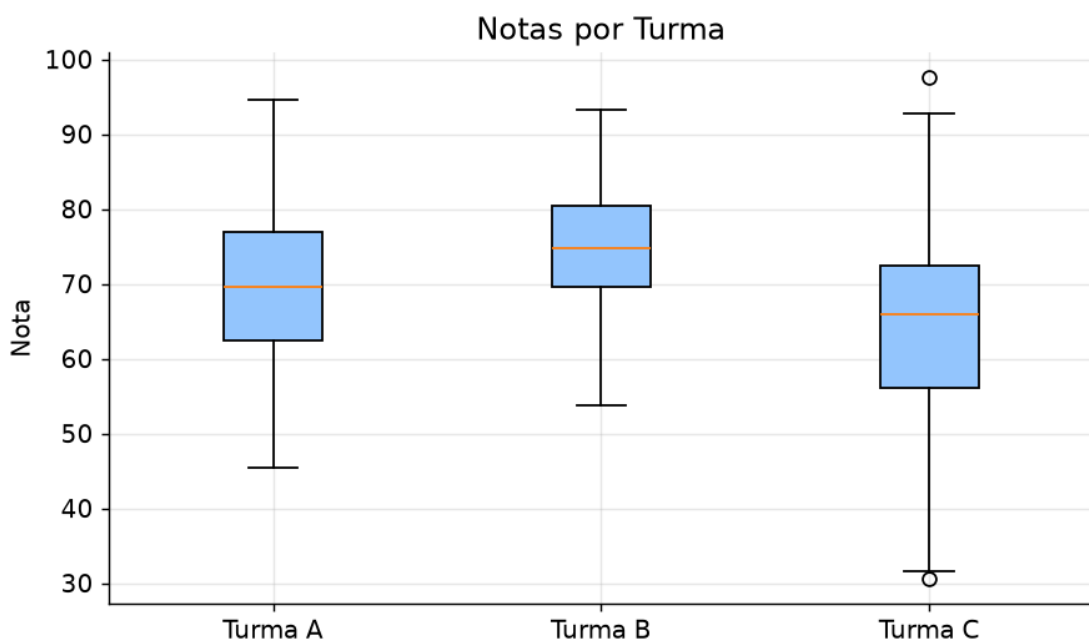


Figura 7 — Boxplot comparando três grupos.

12. Subplots — Múltiplos Gráficos

`plt.subplots(linhas, colunas)` cria uma grade de gráficos em uma única figura.

```
fig, axes = plt.subplots(2, 2, figsize=(8, 5.5))

axes[0,0].plot(x, np.sin(x));          axes[0,0].set_title('Seno')
axes[0,1].plot(x, np.cos(x));          axes[0,1].set_title('Cosseno')
axes[1,0].bar(categorias[:4], vendas[:4]); axes[1,0].set_title('Vendas')
axes[1,1].hist(dados, bins=20);        axes[1,1].set_title('Histograma')

fig.suptitle('Múltiplos Gráficos com subplots')
plt.tight_layout()
plt.show()
```

Múltiplos Gráficos com subplots

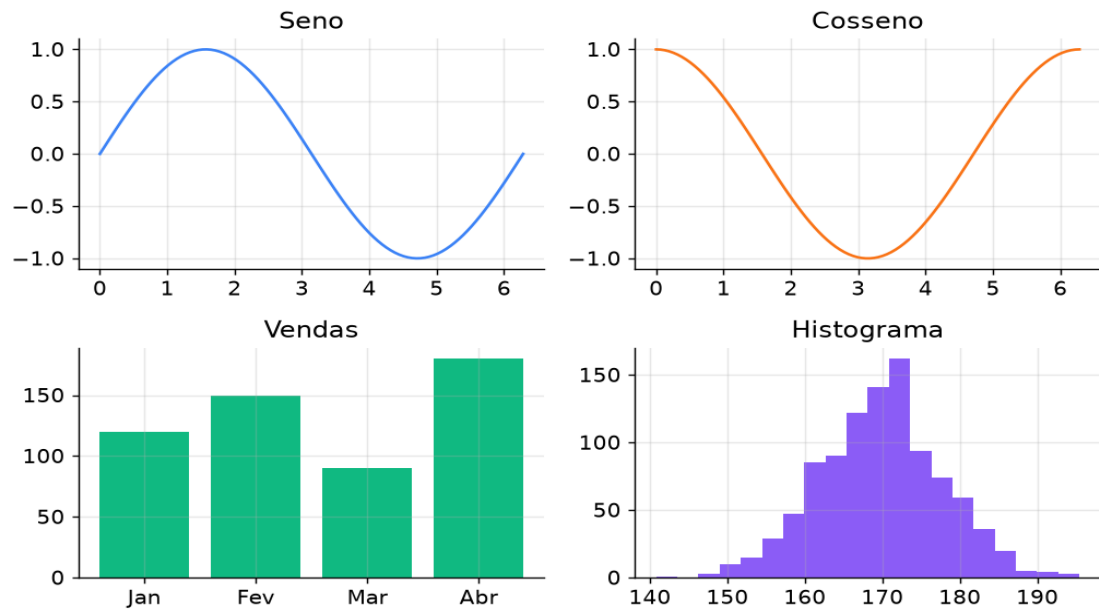


Figura 8 — Grade 2x2 de gráficos.

13. Matplotlib + Pandas

Pandas integra-se diretamente com matplotlib. Você pode passar colunas do DataFrame para as funções de plot.

```
import pandas as pd

df = pd.DataFrame({
    'Mês':      ['Jan', 'Fev', 'Mar', 'Abr', 'Mai', 'Jun'],
    'Receita':  [12000, 15000, 9000, 18000, 21000, 17500],
    'Custo':    [ 8000, 10000, 7500, 12000, 14000, 11000],
})
df['Lucro'] = df['Receita'] - df['Custo']

plt.plot(df['Mês'], df['Receita'], marker='o', label='Receita')
plt.plot(df['Mês'], df['Custo'], marker='s', label='Custo')
plt.fill_between(df['Mês'], df['Custo'], df['Receita'],
                alpha=0.2, color='green')
plt.legend(); plt.ylabel('R$')
plt.title('Receita vs Custo')
plt.show()
```

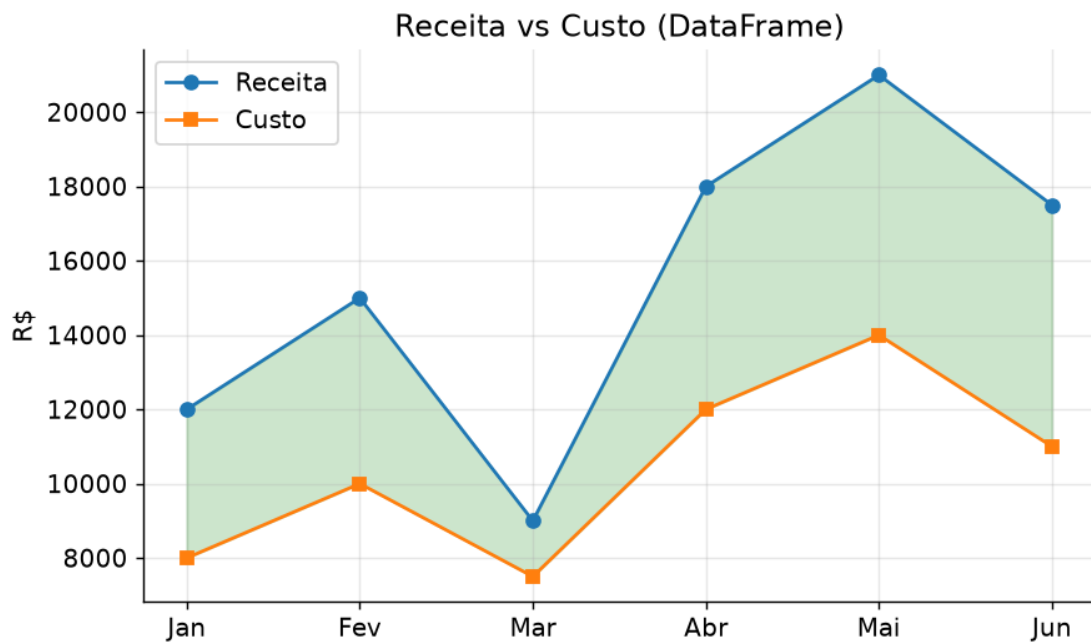


Figura 9 — Plot direto de DataFrame com área preenchida.

14. Personalização e Estilo

Você pode controlar cores, espessuras, anotações, grids e estilos globais.

```
plt.figure()
plt.plot(x, np.sin(x)*np.exp(-x/5),
        color='#EF4444', linewidth=3)
plt.fill_between(x, np.sin(x)*np.exp(-x/5),
                alpha=0.3, color='#EF4444')

plt.title('Oscilação Amortecida',
        fontsize=14, fontweight='bold')
plt.xlabel('Tempo (s)'); plt.ylabel('Amplitude')
# Anotação com seta
```

```
plt.annotate('Pico', xy=(1.5, 0.7), xytext=(3, 0.8),
            arrowprops=dict(arrowstyle='->'))
plt.show()
```

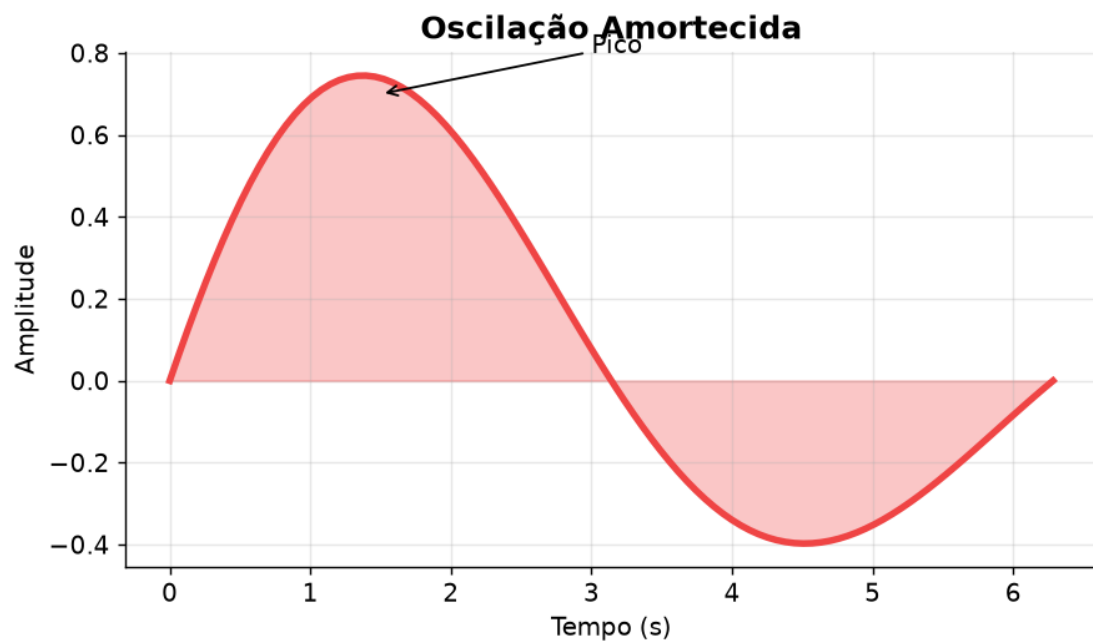


Figura 10 — Personalização com anotações e preenchimento.

Para mudar o tema globalmente, use estilos prontos:

```
plt.style.use('ggplot') # ou 'seaborn-v0_8', 'dark_background', 'classic'
```

15. Salvando Gráficos

Use `plt.savefig` ANTES de `plt.show`.

```
plt.plot(x, np.sin(x))
plt.savefig('grafico.png', dpi=300, bbox_inches='tight')
plt.savefig('grafico.pdf')    # vetorial
plt.savefig('grafico.svg')    # vetorial para web
```

Parâmetros importantes:

- **dpi**: resolução (300 para impressão).
- **bbox_inches='tight'**: remove margens em branco.
- **transparent=True**: fundo transparente (PNG).

16. Conclusão

Você aprendeu a preparar dados, entender a anatomia de uma figura e construir os principais tipos de gráfico no Matplotlib: linha, barras, pizza, dispersão, histograma, boxplot e subplots — além de personalizar e exportar resultados.

Próximos passos sugeridos:

- Explorar a biblioteca **Seaborn** para gráficos estatísticos mais bonitos.
- Estudar **Plotly** para visualizações interativas.
- Praticar com datasets reais (Kaggle, IBGE, dados.gov.br).

Bons gráficos!